

QA en la Construcción de Software

Técnicas y Herramientas para el Aseguramiento de la Calidad

Estándares y Métricas de Calidad en el Desarrollo de Software · Semana 8 · UNIMINUTO

Nota conceptual clave: Técnica y herramienta no son la misma categoría. Una **técnica** es el método o enfoque (qué se hace y con qué objetivo de calidad). Una **herramienta** es el artefacto que automatiza o soporta esa técnica (con qué se implementa). La elección de técnica precede siempre a la elección de herramienta.

1. Pruebas Dinámicas (requieren ejecución del software)

Técnica	Descripción	Herramientas de soporte	Norma / Marco
Pruebas unitarias	Verificación de unidades de código de forma aislada, sin dependencias externas.	JUnit 5, TestNG, NUnit, pytest, Jest	ISO/IEC 25010 · Confiabilidad CMMI PA VER
Pruebas de integración	Verificación de la interacción entre módulos, servicios o capas del sistema.	Spring Boot Test, Testcontainers, REST-assured, WireMock	IEEE Std 829 ISO/IEC 25010 · Funcionalidad
Pruebas de API / servicios REST	Validación de contratos y comportamiento de endpoints REST o SOAP.	Karate Framework, Postman, REST-assured, SoapUI	ISO/IEC 25010 · Adecuación funcional
Pruebas funcionales de UI	Verificación del comportamiento del sistema desde la interfaz de usuario en navegador.	Selenium WebDriver, Playwright, Cypress	IEEE Std 829
Pruebas de regresión	Verificación de que cambios nuevos no rompen funcionalidad existente.	Selenium (suites), JUnit, Karate en pipeline CI	ISO/IEC 25010 · Confiabilidad
Pruebas de aceptación (BDD)	Validación del comportamiento del sistema contra criterios de negocio escritos en lenguaje natural (Gherkin: Given/When/Then).	Cucumber + Selenium, Karate Framework, FitNesse	ISO/IEC 25010 · Adecuación funcional CMMI PA VAL
Pruebas de rendimiento	Medición del comportamiento del sistema bajo carga, estrés y picos de concurrencia.	Apache JMeter, Gatling, k6, Locust	ISO/IEC 25010 · Eficiencia de desempeño
Pruebas de seguridad	Detección activa de vulnerabilidades (SQL injection, XSS, CSRF) en el software construido.	OWASP ZAP, Burp Suite, Snyk, OWASP Dependency-Check	OWASP Top 10 ISO/IEC 25010 · Seguridad
Pruebas de mutación	Evaluación de la efectividad de los casos de prueba introduciendo defectos artificiales (mutantes) en el código.	PIT (Pitest) – Java, Stryker – JavaScript/TypeScript	IEEE Std 829 ISO/IEC 25010 · Confiabilidad
Pruebas de humo (Smoke Testing)	Verificación rápida de que las funciones críticas operan correctamente tras un despliegue. Primera barrera del pipeline.	Scripts JUnit / Postman / Karate en Commit Stage	Buenas prácticas DevOps Humble & Farley (2010)

2. Técnicas Estáticas (no requieren ejecución del software)

Técnica	Descripción	Herramientas de soporte	Norma / Marco
Análisis estático de código	Inspección automatizada del código fuente para detectar bugs, vulnerabilidades y code smells sin ejecutarlo.	SonarQube, SpotBugs, PMD, Checkstyle, ESLint, Pylint	ISO/IEC 25010 · Mantenibilidad · Seguridad

Técnica	Descripción	Herramientas de soporte	Norma / Marco
Revisión de código (Code Review)	Inspección manual del código por pares antes de integrar al repositorio principal.	GitHub Pull Requests, GitLab MR, Gerrit, Crucible	IEEE Std 1028 CMMI PA VER
Revisión Técnica Formal (RTF)	Evaluación estructurada de artefactos (código, diseño, requisitos) con roles definidos: autor, revisor líder, revisores, moderador.	Listas de verificación + JIRA para seguimiento de hallazgos	IEEE Std 1028 CMMI PA VER Marin et al. (2019)
Inspección de Fagan	Variante altamente formal de RTF con seis fases: planificación, presentación, preparación, reunión, retrabajo y seguimiento.	Listas de verificación formales estructuradas	IEEE Std 1028
Análisis de cobertura de código	Medición del porcentaje de código fuente ejecutado por el conjunto de pruebas. Detecta zonas sin cobertura (objetivo: >80% en código crítico).	JaCoCo, Cobertura (Java) · Istanbul/NYC (JS) · SonarQube	ISO/IEC 25010 · Confiabilidad
Linting	Detección automática de errores de estilo, sintaxis y convenciones directamente en el editor o en el pipeline.	ESLint (JS/TS), Pylint (Python), Checkstyle (Java), RuboCop (Ruby)	Guías de estilo del lenguaje ISO/IEC 25010 · Mantenibilidad
Análisis de dependencias y licencias	Detección de dependencias de terceros con vulnerabilidades conocidas (CVE) u obsoletas en el proyecto.	OWASP Dependency-Check, Snyk, Dependabot, Black Duck	OWASP Top 10 ISO/IEC 25010 · Seguridad
Análisis de deuda técnica	Cuantificación del esfuerzo acumulado de remediación en el código (atajos, diseño deficiente, falta de refactoring). Escala A–E.	SonarQube (minutos de remediación), CodeClimate, Codacy	ISO/IEC 25010 · Mantenibilidad

3. Gestión y Trazabilidad del Proceso de QA

Técnica	Descripción	Herramientas de soporte	Norma / Marco
Gestión del ciclo de vida de defectos	Registro, clasificación (severidad/prioridad), asignación, seguimiento y cierre de defectos desde su detección hasta su resolución verificada.	JIRA, Azure DevOps Boards, Bugzilla, Linear	CMMI PA CAR ISO/IEC 25010
Trazabilidad requisito–prueba–defecto	Vinculación explícita y verificable entre cada requisito funcional, sus casos de prueba y los defectos encontrados. Permite auditar cobertura por requisito.	JIRA + Xray, JIRA + Zephyr Scale, TestRail, qTest	IEEE Std 829 CMMI PA VER/VAL ISO/IEC 29110
Gestión de casos de prueba	Diseño, organización, versionado y ejecución controlada de casos de prueba con registro de evidencia de resultados (pass/fail) e historial.	TestRail, Zephyr Scale, Xray, qTest, PractiTest	IEEE Std 829 ISO/IEC 25040
Análisis de causa raíz (RCA)	Identificación de la causa fundamental de un defecto para implementar acciones correctivas que prevengan su recurrencia.	Diagrama de Ishikawa, 5 Porqués, análisis de Pareto · JIRA (seguimiento)	CMMI PA CAR ISO/IEC 25010
Análisis de tendencias de calidad	Seguimiento histórico de métricas (densidad de defectos, cobertura, éxito del pipeline) para identificar patrones de mejora o deterioro.	SonarQube (histórico), JIRA Dashboards, Power BI, Grafana	CMMI PA MA ISO/IEC 25040
Revisiones de adherencia a procesos	Verificación objetiva de que el equipo sigue los procesos, estándares y metodologías definidos. Detecta desviaciones antes de que generen defectos.	Listas de verificación por fase, auditorías internas, reportes de conformidad	CMMI PA PPQA ISO 29110 Marin et al. (2019)

4. Integración de QA en el Pipeline CI/CD

Técnica	Descripción	Herramientas de soporte	Norma / Marco
Integración Continua (CI) con quality gates	Ejecución automática de pruebas y análisis estático en cada commit. El pipeline falla si el código no cumple umbrales definidos (cobertura mínima, cero vulnerabilidades críticas, deuda técnica tolerable).	Jenkins, GitHub Actions, GitLab CI/CD + SonarQube como quality gate	Humble & Farley (2010) Buenas prácticas DevOps
Pruebas continuas (Continuous Testing)	Ejecución de pruebas automatizadas en cada etapa del deployment pipeline como condición de avance: Commit Stage → Acceptance Stage → Performance Stage → Exploratory Stage.	JUnit + Karate + Selenium en pipeline · Gatling para rendimiento	Mascheroni & Irrazábal (2018) ISO/IEC 25010
Shift-left testing	Desplazamiento de las actividades de prueba hacia las fases más tempranas del desarrollo. Las pruebas se diseñan desde los requisitos, no al final del sprint.	Pruebas unitarias desde el día 1 · TDD · RTF de requisitos	ISTQB ISO/IEC 25010 Marin et al. (2019)
Infrastructure as Code (IaC) testing	Verificación automatizada de la configuración de infraestructura (Terraform, Ansible, Docker) antes del despliegue para detectar misconfiguraciones de seguridad o disponibilidad.	Terratest, Checkov, Ansible Molecule, Trivy	Buenas prácticas DevSecOps ISO/IEC 25010 · Seguridad

Puntos para Reforzar en Clase

Regla de selección	La elección de técnica precede siempre a la de herramienta. El flujo correcto es: (1) identificar la característica de calidad a cubrir (ISO/IEC 25010), (2) seleccionar la técnica que mejor la aborda, (3) evaluar herramientas según el stack y la integración con el pipeline, (4) medir con métricas si la técnica reduce el riesgo identificado. Seleccionar SonarQube sin tener claro que se necesita análisis estático es el error más frecuente.
Ninguna técnica es suficiente por sí sola	Las organizaciones que concentran todo el esfuerzo de QA en pruebas dinámicas (etapas finales) tienen costos de corrección significativamente más altos. El Standish Group reporta que el 50% de defectos se introducen en requisitos y el 30% en diseño: etapas donde las pruebas dinámicas no aplican. La combinación de técnicas estáticas desde fases tempranas con pruebas continuas en el pipeline es la estrategia con mayor retorno (Marin, Trujillo y Buedo, 2019; Mascheroni e Irrazábal, 2018).

Referencias

- Mascheroni, M. A. e Irrazábal, E. (2018). Problemas que afectan a la calidad de software en Entrega Continua y Pruebas Continuas. XXIV CACIC, Tandil, Argentina.
- Marin Diaz, A., Trujillo Casañola, Y. y Buedo Hidalgo, D. (2019). Apuntes para gestionar actividades de calidad en proyectos de desarrollo de software. *Ingeniare*, 27(2), 319–327.
- Humble, J. y Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- IEEE Std 829 – Documentación de pruebas de software. | IEEE Std 1028 – Revisiones e inspecciones del software.
- ISO/IEC 25010:2011 – Modelo de calidad del producto (SQuARE). | CMMI-DEV v2.0 – PA: VER, VAL, MA, CAR, PPQA.